

So i have thought this project that exactly does this thing that we will create multiple agents like one agents will trace the logs in your codebase through braintrust evalve model and then it will generate a voice note that could summarized the situation or the problem and then it will prompt the user to ask to make the changes or not then agent will make the changes and make the coebase optimized so is it a good project and can we make this according to hackathons requirement ? and suggest me a architecture

Yes, that's a strong hackathon project, and it fits the requirements if you tighten two things:

1. **Real-time data:** make sure the “logs” come from a real-time source (Datadog logs/metrics stream, or a live tail of logs from a running service), not a static file.
2. **Autonomy + safe action:** the agent should be able to act (create a PR/patch, open an issue, or apply a safe config change) after a quick user confirmation. That still counts as autonomous operation with a human-in-the-loop safety gate.

If you execute it cleanly, this is exactly the kind of “real companies ship” tool.

Sponsor tool mapping (minimum 3)

- **Datadog:** real-time logs/metrics ingestion + anomaly trigger
- **Braintrust:** evaluation harness (quality of root cause, action plan, diff quality, regression risk) and improvement tracking over runs
- **ElevenLabs** (and/or **Modulate** if you also do voice interaction): generate the voice incident summary and “next steps” brief

Optional (if needed): LLM provider sponsor for reasoning if included.

Architecture you can implement fast

1) Data layer (Real-time)

Datadog Ingestor

- Pull logs (or metrics like error rate, p95 latency) on a short interval (ex: every 10–20s)
- Detect incident triggers:
 - “error rate spike”
 - “new exception signature”
 - “latency regression”
- Output: an **Incident Event** object:
 - service
 - time window

- top error fingerprints
- example log lines
- correlated metric snapshot

2) Agent Orchestrator

Incident Orchestrator

- Receives Incident Event
- Spins up a multi-agent workflow (simple state machine)

Think of it like:

- DETECT -> TRIAGE -> DIAGNOSE -> PROPOSE -> CONFIRM -> PATCH -> VERIFY -> LEARN

3) Multi-agent roles (keep them crisp)

Agent A: Triage Agent

- Summarizes what changed and what is breaking
- Clusters logs into 1–3 likely issues (by signature)
- Produces a short incident brief

Agent B: Codebase Tracer Agent

- Uses the error signature + stack traces + grep/ripgrep over repo
- Maps logs to code locations
- Produces “suspect files/functions” and hypotheses

Agent C: Fix/Optimization Agent

- Generates a minimal patch:
 - fix bug, add guard, adjust query, add caching, reduce overfetch, etc.
- Keeps patch small and reversible
- Outputs a diff

Agent D: Verification Agent

- Runs tests/lint (or a lightweight “smoke check” if tests are heavy)
- Verifies the fix doesn’t break build
- Produces result summary

4) Evaluation and self-improvement loop (Braintrust)

Braintrust Eval Runner

Evaluate each run on:

- Root-cause correctness (does it match log evidence?)
- Patch quality (minimal, safe, coherent)
- Regression risk (touches too many files? no tests?)
- Time-to-resolution / number of steps
- User satisfaction signal (did user accept patch?)

Self-improvement mechanism

Store a “playbook memory”:

- incident signature → best fix pattern
- which heuristics worked (tool order, prompts, narrowing strategy)
- which fixes failed

Next incident with similar signature:

- reuse the best playbook
- adjust thresholds and prompt templates
- reduce number of steps and verbosity

This is what makes it a *self-improving agent* and not just an assistant.

5) Voice layer (ElevenLabs / Modulate)

Voice Brief Generator

- Converts the triage summary into a 15–25s voice note:
 - what happened
 - impact
 - likely root cause
 - proposed fix + risk
 - ask for confirmation

Optionally:

- If using Modulate for interactive voice, let the user respond “yes/no” verbally.
- If not, user clicks “Apply Fix” in your chat UI.

6) Action layer (safe changes)

Patch Applier

Pick the safest “action” for hackathon time:

- Generate a git diff + open a PR (best)

- Or apply patch locally and show the diff
- Or create a “suggested change” file and let user apply

In the demo, show:

- diff
 - tests passing
 - before/after metric improvement (even if simulated)
-

How to make it clearly satisfy the hackathon requirements

- **Real-time data:** show a live Datadog log/metric feed and an incident trigger happening “now”
 - **Autonomy:** agent starts triage automatically on trigger, no user prompt needed
 - **Self-improvement:** run 2 incidents:
 - incident #1: agent takes 2 iterations, Braintrust score 0.62
 - incident #2 (similar): agent uses stored playbook, 1 iteration, score 0.84
 - **3 sponsor tools:** explicitly show Datadog + Braintrust + ElevenLabs in the demo
-

Minimal build plan (so it’s shippable)

If time is tight, cut scope like this:

- Real-time source: Datadog logs only (skip metrics correlation)
- “Fix” action: generate a patch + run 1 command (tests/lint) + show diff
- Memory: simple JSON store keyed by error signature
- Voice: only incident summary + confirmation prompt

That will still look like a production agent.

If you tell me which language your backend is (Node/Python) and how you plan to “apply changes” (PR vs local patch), I’ll give you a concrete folder structure + API routes to plug into your Next.js chat UI.

Yep, understood.

Plan:

1. Build the **Python backend + Streamlit demo UI first** (fast iteration).
2. If time remains, swap Streamlit UI for your **Next.js chat UI** by calling the same backend endpoints.

Below is a clean, hackathon-friendly **folder structure + scaffolding** that supports:

- Datadog real-time log pull
 - Multi-agent pipeline (triage → trace → patch → verify)
 - Braintrust eval loop
 - ElevenLabs voice summary
 - Local patch generation + apply
 - Streamlit demo
-

Folder structure

```
voiceops/  
  README.md  
  .env.example  
  requirements.txt  
  
app/  
  streamlit_app.py  
  
src/  
  config.py  
  main.py  
  
core/  
  models.py  
  memory.py  
  orchestrator.py  
  tools.py  
  patcher.py  
  verifier.py  
  
integrations/  
  datadog_client.py  
  braintrust_eval.py  
  elevenlabs_tts.py
```

llm.py

agents/
 triage_agent.py
 trace_agent.py
 fix_agent.py
 verify_agent.py

data/
 memory.json
runs/

repo_target/
 # optional: clone/copy a small buggy sample repo here for demo

requirements.txt (minimal, practical)

streamlit==1.32.0
python-dotenv==1.0.1
requests==2.31.0

pydantic==2.6.1
rich==13.7.0

gitpython==3.1.42

If you already know you will use an LLM SDK, add it later. For speed, you can call an LLM via [requests](#) in [integrations/llm.py](#).

.env.example

Datadog
DD_API_KEY=...
DD_APP_KEY=...
DD_SITE=datadoghq.com

Braintrust
BRAINTRUST_API_KEY=...
BRAINTRUST_PROJECT=voiceops

```
# ElevenLabs
ELEVENLABS_API_KEY=...
ELEVENLABS_VOICE_ID=...

# LLM (choose one you have access to)
LLM_PROVIDER=openai
LLM_API_KEY=...
LLM_MODEL=gpt-4.1-mini
```

Core data models (src/core/models.py)

```
from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any
```

```
class LogEvent(BaseModel):
    timestamp: str
    service: str
    message: str
    attributes: Dict[str, Any] = Field(default_factory=dict)

class Incident(BaseModel):
    id: str
    created_at: str
    service: str
    summary: str
    log_fingerprints: List[str] = Field(default_factory=list)
    sample_logs: List[LogEvent] = Field(default_factory=list)
    severity: str = "medium"

class TraceResult(BaseModel):
    suspected_files: List[str] = Field(default_factory=list)
    suspected_functions: List[str] = Field(default_factory=list)
    hypothesis: str

class PatchProposal(BaseModel):
    title: str
    rationale: str
    diff: str # unified diff text
    files_touched: List[str] = Field(default_factory=list)
    risk: str = "low"
```

```
class VerificationResult(BaseModel):
    ok: bool
    command: str
    output: str
```

Orchestrator state machine (src/core/orchestrator.py)

```
from dataclasses import dataclass
from typing import Optional, Dict, Any
from .models import Incident, TraceResult, PatchProposal, VerificationResult
```

```
@dataclass
```

```
class RunContext:
    repo_path: str
    incident: Incident
    trace: Optional[TraceResult] = None
    patch: Optional[PatchProposal] = None
    verification: Optional[VerificationResult] = None
    eval_scores: Dict[str, Any] = None
    voice_path: Optional[str] = None
```

```
class Orchestrator:
```

```
    def __init__(self, triage_agent, trace_agent, fix_agent, verify_agent, evaluator, tts, memory):
        self.triage_agent = triage_agent
        self.trace_agent = trace_agent
        self.fix_agent = fix_agent
        self.verify_agent = verify_agent
        self.evaluator = evaluator
        self.tts = tts
        self.memory = memory
```

```
    def run(self, ctx: RunContext, auto_apply: bool = False) -> RunContext:
```

```
        # 1) Triage (already have incident; triage agent can enrich summary)
        ctx.incident = self.triage_agent.enrich_incident(ctx.incident)
```

```
        # 2) Trace
```

```
        ctx.trace = self.trace_agent.trace(ctx.repo_path, ctx.incident)
```

```
        # 3) Propose patch (use memory playbook)
```

```
        playbook = self.memory.get_playbook(ctx.incident)
```

```
        ctx.patch = self.fix_agent.propose_patch(ctx.repo_path, ctx.incident, ctx.trace,
        playbook=playbook)
```

```

# 4) Generate voice brief
voice_text = self._voice_brief_text(ctx)
ctx.voice_path = self.tts.synthesize_to_file(voice_text)

# 5) Evaluate proposal quality (Braintrust)
ctx.eval_scores = self.evaluator.evaluate(ctx)

# 6) Optionally apply patch + verify
if auto_apply:
    self.fix_agent.apply_patch(ctx.repo_path, ctx.patch.diff)
    ctx.verification = self.verify_agent.verify(ctx.repo_path)

# 7) Re-evaluate with verification signals + learn
ctx.eval_scores = self.evaluator.evaluate(ctx)
self.memory.learn(ctx)

return ctx

def _voice_brief_text(self, ctx: RunContext) -> str:
    return (
        f"Incident detected in {ctx.incident.service}. "
        f"Summary: {ctx.incident.summary}. "
        f"Hypothesis: {ctx.trace.hypothesis if ctx.trace else 'unknown'}. "
        f"Proposed fix: {ctx.patch.title if ctx.patch else 'none'}. "
        f"Risk: {ctx.patch.risk if ctx.patch else 'unknown'}. "
        f"Please confirm if you want to apply the patch."
    )

```

Memory (self-improvement) (src/core/memory.py)

```

import json
from pathlib import Path
from typing import Dict, Any
from .models import Incident

class MemoryStore:
    def __init__(self, path: str):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        if not self.path.exists():
            self.path.write_text(json.dumps({"playbooks": {}, "history": []}, indent=2))

```

```

def _load(self) -> Dict[str, Any]:
    return json.loads(self.path.read_text())

def _save(self, data: Dict[str, Any]) -> None:
    self.path.write_text(json.dumps(data, indent=2))

def fingerprint(self, incident: Incident) -> str:
    # Simple stable fingerprint for hackathon: service + top fingerprint
    top = incident.log_fingerprints[0] if incident.log_fingerprints else "unknown"
    return f"{incident.service}:{top}"

def get_playbook(self, incident: Incident) -> Dict[str, Any]:
    data = self._load()
    fp = self.fingerprint(incident)
    return data["playbooks"].get(fp, {})

def learn(self, ctx) -> None:
    data = self._load()
    fp = self.fingerprint(ctx.incident)

    # Minimal learning: store last successful patch title + reasoning
    if ctx.verification and ctx.verification.ok:
        data["playbooks"][fp] = {
            "best_patch_title": ctx.patch.title,
            "best_rationale": ctx.patch.rationale,
            "files_touched": ctx.patch.files_touched,
            "last_eval": ctx.eval_scores,
        }

    data["history"].append({
        "incident_fp": fp,
        "patch_title": ctx.patch.title if ctx.patch else None,
        "verification_ok": bool(ctx.verification.ok) if ctx.verification else None,
        "eval": ctx.eval_scores,
    })
    self._save(data)

```

Local patching utilities (src/core/patcher.py)

```

import subprocess
from pathlib import Path

```

```
from typing import Optional
```

```
class LocalPatcher:
```

```
    def apply_unified_diff(self, repo_path: str, diff_text: str) -> None:
```

```
        # safest hackathon approach: use `git apply`
```

```
        repo = Path(repo_path)
```

```
        proc = subprocess.run(
```

```
            ["git", "apply", "-"],
```

```
            input=diff_text.encode("utf-8"),
```

```
            cwd=str(repo),
```

```
            capture_output=True
```

```
        )
```

```
        if proc.returncode != 0:
```

```
            raise RuntimeError(f"git apply failed:\n{proc.stderr.decode('utf-8')}")
```

Verifier (src/core/verifier.py)

```
import subprocess
```

```
from .models import VerificationResult
```

```
class Verifier:
```

```
    def __init__(self, command: str = "pytest -q"):
```

```
        self.command = command
```

```
    def verify(self, repo_path: str) -> VerificationResult:
```

```
        proc = subprocess.run(
```

```
            self.command.split(),
```

```
            cwd=repo_path,
```

```
            capture_output=True,
```

```
            text=True
```

```
        )
```

```
        return VerificationResult(
```

```
            ok=(proc.returncode == 0),
```

```
            command=self.command,
```

```
            output=(proc.stdout + "\n" + proc.stderr).strip()
```

```
        )
```

Streamlit UI (app/streamlit_app.py)

This gives you a clean demo:

- Pull logs (or simulate)
- Run pipeline
- Play voice note
- Show diff
- Button: “Apply patch”
- Button: “Verify”
- Show Braintrust eval score change

```

import streamlit as st
from datetime import datetime
from src.core.orchestrator import Orchestrator, RunContext
from src.core.memory import MemoryStore
from src.core.verifier import Verifier
from src.core.patcher import LocalPatcher

# integrations (you will implement lightweight clients)
from src.integrations.datadog_client import DatadogClient
from src.integrations.braintrust_eval import BraintrustEvaluator
from src.integrations.elevenlabs_tts import ElevenLabsTTS
from src.integrations.llm import LLMClient

from src.agents.triage_agent import TriageAgent
from src.agents.trace_agent import TraceAgent
from src.agents.fix_agent import FixAgent
from src.agents.verify_agent import VerifyAgent

st.set_page_config(page_title="VoiceOps", layout="wide")
st.title("VoiceOps: Self-Improving Voice Incident Agent")

repo_path = st.text_input("Repo path", value="./repo_target")
service = st.text_input("Service name", value="api")
time_window = st.slider("Lookback minutes", 1, 60, 10)

if "ctx" not in st.session_state:
    st.session_state.ctx = None

# Setup dependencies
dd = DatadogClient()
llm = LLMClient()
tts = ElevenLabsTTS()
memory = MemoryStore("data/memory.json")
evaluator = BraintrustEvaluator()
verifier = Verifier(command="pytest -q")
patcher = LocalPatcher()

```

```

triage_agent = TriageAgent(llm=llm)
trace_agent = TraceAgent(llm=llm)
fix_agent = FixAgent(llm=llm, patcher=patcher)
verify_agent = VerifyAgent(verifier=verifier)

orch = Orchestrator(
    triage_agent=triage_agent,
    trace_agent=trace_agent,
    fix_agent=fix_agent,
    verify_agent=verify_agent,
    evaluator=evaluator,
    tts=tts,
    memory=memory
)

col1, col2 = st.columns([1, 1])

with col1:
    st.subheader("1) Real-time Logs")
    if st.button("Fetch latest logs"):
        incident = dd.fetch_incident(service=service, minutes=time_window)
        st.session_state.incident = incident
        st.success("Incident event created from real-time logs.")
    if "incident" in st.session_state:
        st.json(st.session_state.incident.model_dump())

with col2:
    st.subheader("2) Agent Run")
    auto_apply = st.checkbox("Auto-apply patch (demo mode)", value=False)
    if st.button("Run agents"):
        if "incident" not in st.session_state:
            st.error("Fetch logs first.")
        else:
            ctx = RunContext(
                repo_path=repo_path,
                incident=st.session_state.incident
            )
            ctx = orch.run(ctx, auto_apply=auto_apply)
            st.session_state.ctx = ctx
            st.success("Run complete.")

if st.session_state.ctx:
    ctx = st.session_state.ctx

```

```

st.divider()
c1, c2, c3 = st.columns([1, 1, 1])

with c1:
    st.subheader("Voice Brief (ElevenLabs)")
    if ctx.voice_path:
        audio_bytes = open(ctx.voice_path, "rb").read()
        st.audio(audio_bytes, format="audio/mpeg")

with c2:
    st.subheader("Patch Proposal")
    st.write(ctx.patch.title if ctx.patch else "No patch")
    if ctx.patch:
        st.code(ctx.patch.diff, language="diff")

    if st.button("Apply patch now"):
        fix_agent.apply_patch(repo_path, ctx.patch.diff)
        st.success("Patch applied locally.")

with c3:
    st.subheader("Evaluation (Braintrust)")
    st.json(ctx.eval_scores or {})

st.divider()
st.subheader("Verification")
if st.button("Run tests"):
    vr = verifier.verify(repo_path)
    st.write("OK" if vr.ok else "FAILED")
    st.code(vr.output)

```

What to implement next (in this exact order)

1. `integrations/datadog_client.py`
 - Either real Datadog API pull, or a fallback simulator if keys fail.
 - Output an `Incident` object.
2. `integrations/elevenlabs_tts.py`
 - Take text → mp3 file saved to `data/runs/<run_id>.mp3`
3. `integrations/braintrust_eval.py`
 - Post run artifacts to Braintrust and return a numeric score.
4. Agent stubs:

- `triage_agent.py` (summarize)
- `trace_agent.py` (map logs to files via grep + LLM)
- `fix_agent.py` (produce unified diff)
- `verify_agent.py` (wrap verifier)

If you do those, the rest composes naturally.

Quick “definition of done” for hackathon demo

- You click “Fetch latest logs” (Datadog real-time)
 - You click “Run agents”
 - It generates:
 - incident summary
 - suspected code areas
 - patch diff
 - voice note
 - Braintrust score
 - You click “Apply patch”
 - You click “Run tests”
 - Memory learns and next run shows better score or fewer steps
-

If you want, paste your target repo language (Python/Node) and how you plan to generate diffs (LLM writes diff vs templated fixes). I'll give you the exact `FixAgent` strategy that is least likely to break in a live demo.